

Agentic Healthcare Assistant for Medical Task Automation

A LangGraph plan-and-execute agent that books appointments, manages medical records, summarises patient histories and answers disease questions from MedlinePlus and WHO, with RAG, long-term memory, guard-rails, an evaluation harness and a Streamlit dashboard.

1.00 Medical answers graded correct (QAEvalChain)	0.99 Answer groundedness in retrieved sources	0.96 Planner tool-selection F1 (15 multi-step queries)	1.00 Booking reliability (incl. race & idempotency)
---	---	--	---

Live-LLM evaluation run eval-20260928-075859 · model meta-llama/llama-3.3-70b-instruct via OpenRouter.

Submitted by: Narayanan

Date: September 2026

Deliverables: source code (healthcare_agent/), Streamlit app (app.py), executed notebook, 27-test pytest suite, evaluation reports.

Contents

1. Executive summary
 2. Problem and objectives
 3. Solution architecture
 4. Data and ingestion
 5. Tools, memory and RAG
 6. Planning, prompt engineering and task chaining
 7. Agent execution: the reference scenario
 8. Guard-rails
 9. LLMOps: evaluation and monitoring
 10. Streamlit dashboard
 11. Engineering issues found and fixed
 12. Limitations and future work
- Appendix: running the project

1. Executive summary

Patient-care administration is split across scheduling systems, electronic health records and reference websites. This project builds a **virtual medical assistant** that coordinates them from a single natural-language request. Given the capstone's reference request, *"My 70-year-old father has chronic kidney disease. I want to book a nephrologist for him. Also, can you summarize latest treatment methods?"*, the agent performs four steps in order:

- Identifies the father through the caregiver link.
- Retrieves and summarises his CKD history, with rule-based lab alerts.
- Books the earliest nephrology slot.
- Returns a grounded, cited treatment summary from MedlinePlus and WHO.

The whole run takes about 15–20 seconds, mostly the live web fetch. The system is built on **LangGraph** with a structured-output planner, 13 tools, FAISS long-term memory and a RAG pipeline. It also has deterministic safety, privacy and role guard-rails, and an LLMOps layer covering QAEvalChain grading, offline metrics and per-tool logging.

Key results (live LLM mode)

- **Medical QA:** QAEvalChain rated 100% of answers correct, with 100% key-fact coverage and 0.99 groundedness. Every answer cites its sources.
- **Planner:** tool-selection F1 of 0.96 and exact plan match of 0.80. Specialty routing and patient identification were 100% correct; the plan mismatches are defensible judgment calls (Section 9).
- **Booking:** 100% expected outcomes, constraint satisfaction, idempotency and race safety. Impossible requests return alternatives instead of silently booking something else.
- **History summaries:** QAEvalChain rated 80% correct against the reference summaries in `records.xlsx`.
- **Graceful degradation:** the whole system runs without an API key in a deterministic offline mode, and a hard deadline turns a stalled provider into a logged fallback instead of a hang.

2. Problem and objectives

The problem statement asks for an agentic system that uses LLMs, RAG and memory modules to:

- **Book medical appointments:** automate slot discovery and scheduling from patient intent and doctor availability.
- **Manage medical records:** let attendants add or update structured and unstructured history.
- **Retrieve medical histories:** summarise past diagnoses, treatments and relevant alerts.
- **Search medical information:** fetch up-to-date disease information from trusted sources (MedlinePlus, WHO).

Part 1 covers the agent design: planning and goal decomposition, tools and memory, prompt engineering and task chaining, and the sample execution flow. Part 2 covers LLMOps: evaluation with QAEvalChain, per-module performance logging, and a Streamlit dashboard with patient/doctor views, real-time appointment tracking, retrieved medical information, metrics, memory traces and interactive scenario testing.

Requirement coverage

Requirement	Implementation
Planner decomposes multi-step queries	LLM structured-output <i>Plan</i> (sub-goals, tool, args, dependencies) + validation/repair layer
Appointment booking API	Doctor Schedule API over SQLite slots: atomic, idempotent booking; cancel; reschedule
EHR / patient DB	SQLite EHR built from the dataset; staff-only add/update/free-text note tools
Disease search (web / Medline)	MedlinePlus Web Service + WHO fact sheets → FAISS → cited answer
FAISS store; long-term memory	patient_memory and medical_knowledge indexes; memories recalled into prompts
Structured prompts & chains	Planner → history summary / medical RAG → synthesis → memory extraction
QAEvalChain; per-module metrics	Evaluation harness with six modules; results in SQLite and JSON
Streamlit views, tracking, logs	8-page dashboard incl. memory traces, planning breakdowns, scenario tester

3. Solution architecture

The assistant is a **plan-and-execute** agent implemented as a LangGraph state graph. Each request passes through six nodes; the executor loops once per planned step. A checkpointer persists state per chat thread, which gives short-term memory across turns.

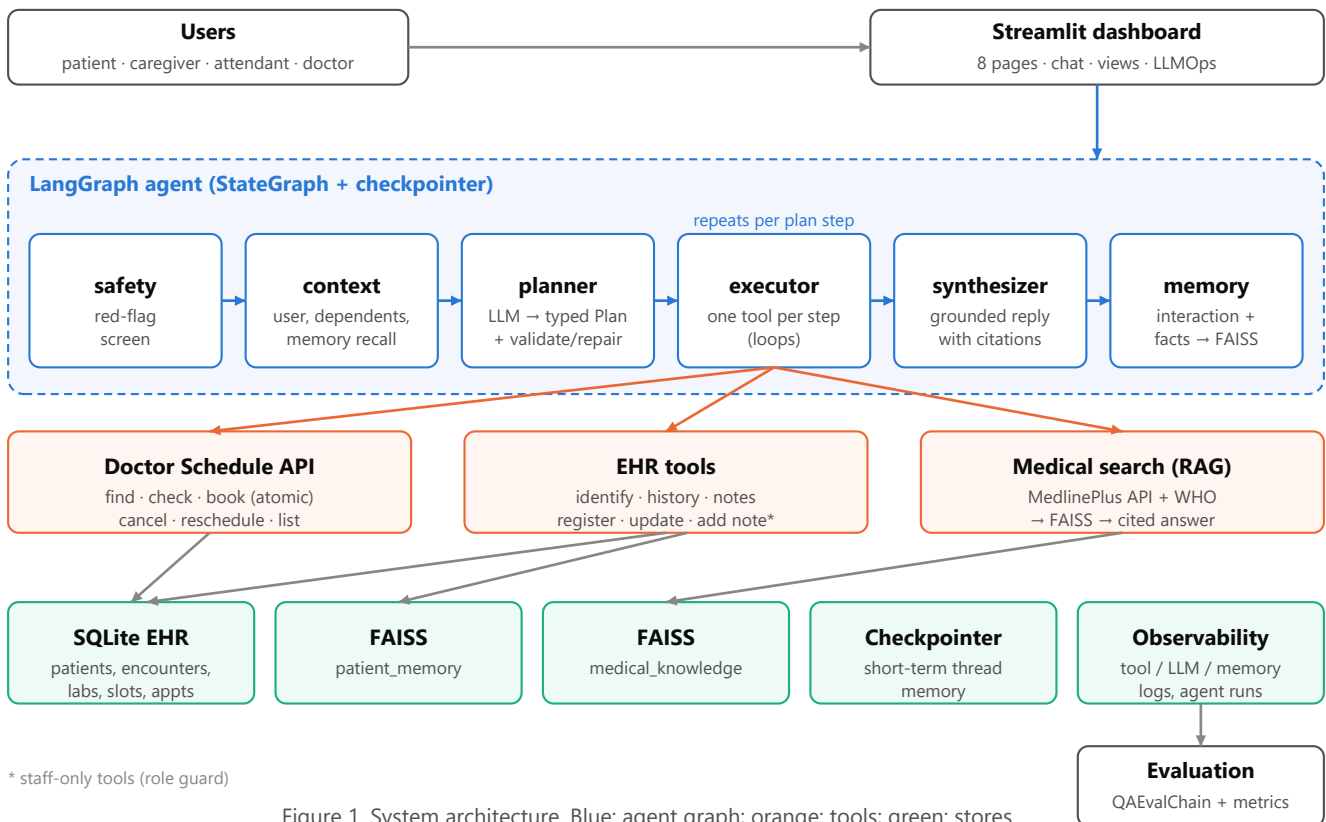


Figure 1. System architecture. Blue: agent graph; orange: tools; green: stores.

Node	Responsibility
safety	Deterministic red-flag screen (heart attack, stroke, breathing difficulty, self-harm...). Emergency guidance is always shown first; background information search is deferred.
context	Resolves the acting user and linked dependents, the active patient from the thread, and recalls long-term memories from FAISS.
planner	LLM produces a typed Plan. The validator drops unknown/duplicate tools, removes staff-only steps for patients, inserts a missing identify step and wires dependencies.
executor	Runs one tool per graph step, injects the resolved patient id and patient context, and skips steps whose dependencies failed.
synthesizer	Writes the reply from tool results only: patient context first, citations kept, failures stated plainly.
memory	Stores an interaction summary and durable facts (conditions, preferences) against the patient.

Design decisions

- **Plan-and-execute over a free ReAct loop.** Requests are compositional and have side effects (a booking must not repeat). An explicit plan is visible in the UI, measurable in evaluation and bounded to one tool call per step.
- **Validate every plan.** LLM plans are proposals; privacy and role rules are enforced in code, so they hold even if the model ignores the prompt.
- **Deterministic where correctness matters.** Clinical alerts, access control, the emergency screen and booking logic are rules. The LLM handles language: planning, summarising, extraction and grounded answering.
- **Provider-agnostic, with fallbacks.** The app needs only structured() and text() calls. Groq, OpenAI, Anthropic, OpenRouter and Ollama are supported, and every call site has an offline fallback.

4. Data and ingestion

The provided dataset contains records.xlsx and four clinical PDFs in two different layouts. Custom parsers convert them into a relational EHR.

Source	Processing	Result
records.xlsx (7 rows)	One row per encounter; de-duplicated by name and phone. The Summary column is kept as evaluation ground truth.	5 patients
3 × History & Physical PDFs	Labelled-field parser: demographics, SOAP sections, ICD-10 diagnoses, medications from plan text, vitals.	3 encounters
sample_patient.pdf (12 pages)	Pages grouped by encounter footer: progress notes, past medical history, and CCD sections with lab results and reference ranges.	2 encounters, 45 labs with H/L flags, meds, vitals
Synthetic (tagged synthetic_demo)	15 doctors in 13 specialties with working hours; rolling 21-day slot calendar (~30% held by other channels); Suresh Negi, 70, CKD 3b, father of Rahul Negi.	Needed by the reference scenario

Why synthetic data was added. The dataset has no doctor directory or calendar, and the reference scenario's father is not in it. Rather than hard-code the scenario, a realistic schedule and one demo patient were generated, and every synthetic row is labelled so it is never confused with the provided records.

Lab results are flagged against their reference ranges (for example ALT 52 U/L vs 6–29 → HIGH; HDL 35 mg/dL vs ≥ 50 → LOW). These flags drive deterministic clinical alerts, together with allergies, elevated blood pressure, polypharmacy, age-related risk and overdue follow-ups.



Patient view

Patient

Suresh Negi (P006)

Age / sex

70 / Male

Active diagnoses

4

Active medications

4

Alerts

9

+91-98110-45671 · 🏠 · 📍 Chattarpur, New Delhi · source: synthetic_demo · caregivers: Rahul Negi (patient is their father)

Clinical alerts

- **HIGH** · allergy - Allergy: sulfa drugs (rash)
- **MEDIUM** · lab - BUN HIGH: 32 mg/dL (ref 7-25, 2026-08-13)
- **MEDIUM** · lab - Creatinine HIGH: 1.9 mg/dL (ref 0.70-1.30, 2026-08-13)
- **MEDIUM** · lab - HbA1c HIGH: 7.4 % (ref <5.7, 2026-08-13)
- **MEDIUM** · lab - Hemoglobin LOW: 11.2 g/dL (ref 13.5-17.5, 2026-08-13)
- **MEDIUM** · lab - Potassium HIGH: 5.4 mmol/L (ref 3.5-5.3, 2026-08-13)
- **MEDIUM** · lab - Urine albumin/creatinine ratio HIGH: 180 mg/g (ref <30, 2026-08-13)
- **MEDIUM** · lab - eGFR LOW: 36 mL/min/1.73m2 (ref > OR = 60, 2026-08-13)
- **LOW** · risk - Geriatric patient (age 70): review dosing and fall risk

AI summary

Encounters

Diagnoses & meds

Labs & vitals

Appointments

Memory

Lab trend

Creatinine

Figure 2. Patient view for the reference patient: rule-based clinical alerts computed from the EHR.

5. Tools, memory and RAG

Group	Tools	Notes
Doctor Schedule API	find_doctors, check_availability, book_appointment, cancel_appointment, reschedule_appointment, list_appointments	Atomic conditional UPDATE on the slot; idempotent per specialty; reschedule books the new slot before releasing the old one; natural-language dates ("next Monday morning").
EHR	identify_patient, get_patient_history, search_patient_notes, register_patient*, update_patient_record*, add_clinical_note*	Fuzzy name match, relation lookup ("my dad"), LLM extraction of structure from free-text notes. * staff only.
Medical search	medical_info_search	MedlinePlus Web Service (XML) + WHO site-restricted search and fact-sheet extraction; cached 43-document seed corpus for offline use.

Every tool is wrapped by a tracing decorator that logs arguments, result, success and latency. A tool reports a business failure (no slot, not authorised) as {ok: false, error} so the executor and synthesizer can react honestly.

Memory

Type	Mechanism	Used for
Short-term	LangGraph checkpointer keyed by chat thread (messages, active patient)	Follow-ups such as "also book a dietitian for him"
Long-term	SQLite memories + FAISS patient_memory (profiles, encounter chunks, learned facts)	Recalled into the planner prompt and history summaries
Knowledge	FAISS medical_knowledge (chunked MedlinePlus/WHO documents)	RAG context

RAG pipeline

Query → live fetch from MedlinePlus and WHO → cache in SQLite → paragraph-aware chunking → bge-small-en-v1.5 embeddings (normalised, cosine via IndexFlatIP) → top-k retrieval with at most two passages per document → numbered-source prompt → answer with inline [n] citations and a patient-specific note. If the network is unavailable, retrieval runs over the cached corpus; with no LLM, a Maximal Marginal Relevance extractive summary is used.

6. Planning, prompt engineering and task chaining

Each sub-task has its own structured prompt (role, inputs, rules, output contract). The prompts form a chain in which the output of one stage becomes context for the next:

Stage	Prompt design
Planner	Tool catalogue; seven planning rules (identify first, history before condition-driven bookings, specialty routing, minimal plans, staff-only record changes, clarification when ambiguous); injected date, user, dependents, active patient, recalled memories and recent turns.
History summary	Fixed sections (overview, diagnoses and treatments, medications, alerts, focus); “use only the record”; keep ICD-10 codes and dates; alerts supplied by rules, not generated.
Medical RAG	Numbered excerpts; inline citations; personalisation note using the patient context produced by the history stage; say what is missing instead of guessing.
Synthesis	Patient-context section first; one heading per task in request order; exact booking details; honest failures; keep citations; under 350 words.
Extraction	Typed schemas for free-text clinical notes and for durable memory facts.

Plan produced by the LLM for the reference request

1. identify_patient(name="Suresh Negi", relation="father", age=70)
2. get_patient_history(focus="chronic kidney disease") ← depends on 1
3. book_appointment(specialty="Nephrology", condition="chronic kidney disease") ← depends on 1, 2
4. medical_info_search(query="latest treatment methods for chronic kidney disease")

The patient's name was resolved from the injected dependents list, a memory lookup in the prompt. The validator confirms dependencies and role permissions before any tool runs.

7. Agent execution: the reference scenario

The problem statement's expected workflow was (1) identify patient and context, (2) retrieve the father's history, (3) query the doctor calendar and book, (4) search and summarise treatments via RAG. The live run below follows it exactly: 4/4 steps succeeded.



Agentic Healthcare Assistant

Signed in as Rahul Negi (patient). Ask for bookings, medical history, record updates or disease information - multi-part requests are decomposed into sub-goals automatically.

My 70-year-old father has chron...

Also book a dietitian for him ne...

Show his upcoming appointme...

What are the symptoms of type...



My 70-year-old father has chronic kidney disease. I want to book a nephrologist for him. Also, can you summarize latest treatment methods?



✓ Done in 19.2s - 4/4 steps succeeded

Patient Context

Suresh Negi, a 70-year-old male, has chronic kidney disease stage 3b, hyperkalemia, type 2 diabetes mellitus with diabetic chronic kidney disease, and hypertensive chronic kidney disease. He is currently taking Empagliflozin, Metformin, Losartan, and Atorvastatin. Key alerts include an allergy to sulfa drugs and abnormal lab results such as high BUN, creatinine, HbA1c, potassium, and urine albumin/creatinine ratio, as well as low hemoglobin and eGFR.

Booking Confirmation

An appointment with Dr. Priya Raman, a nephrologist, has been booked for Suresh Negi. The appointment details are as follows:

- Doctor: Dr. Priya Raman
- Specialty: Nephrology
- Date and Time: 2026-09-28 10:30:00
- Location: Sunrise Multispeciality Clinic, Chennai
- Appointment ID: APT-58874B63

Latest Treatment Methods for Chronic Kidney Disease

The latest treatment methods for chronic kidney disease focus on slowing disease progression, reducing cardiovascular risk, and preventing complications of advanced kidney disease [1]. This includes medications to manage blood pressure, blood sugar, and cholesterol, as well as lifestyle changes such as choosing foods with less salt, controlling blood pressure, keeping blood sugar in target range, limiting alcohol intake, and being physically active [1]. Regular monitoring with eGFR and albuminuria is also crucial for early diagnosis and treatment [2]. For Suresh Negi, working with his healthcare provider to manage his conditions and making lifestyle changes to slow disease progression and reduce cardiovascular risk is essential.

Sources:

1. MedlinePlus (NIH/NLM) - [Chronic Kidney Disease](#)
2. World Health Organization - [Kidney disease](#)
3. World Health Organization - [Guideline for the pharmacological treatment of hypertension in adults](#)

Plan (live) - 4 sub-goals

Intent: Book a nephrologist for the user's father and summarize latest treatment methods for chronic kidney disease

1. **Identify the patient** → `identify_patient` `{"name": "Suresh Negi", "relation": "father", "age": 70}`
2. **Retrieve the patient's medical history** → `get_patient_history` `{"focus": "chronic kidney disease"}` - depends on [1]
3. **Book a nephrologist appointment** → `book_appointment` `{"specialty": "Nephrology", "condition": "chronic kidney disease", "reason": "follow-up for chronic kidney disease"}` - depends on [1, 2]
4. **Summarize latest treatment methods for chronic kidney disease** → `medical_info_search` `{"query": "latest treatment methods for chronic kidney disease"}`

Execution trace - 4/4 steps succeeded, 19168 ms

Figure 3. Live run of the reference scenario: patient context, booking confirmation, cited treatment summary and the planner's decomposition.

Follow-up turns use short-term memory: *"Also book a dietitian for him next week in the morning"* resolves *"him"* to the active patient and books a Nutrition & Dietetics slot the following week before noon. The trace of any run can be replayed, including graph path timings and memory operations:

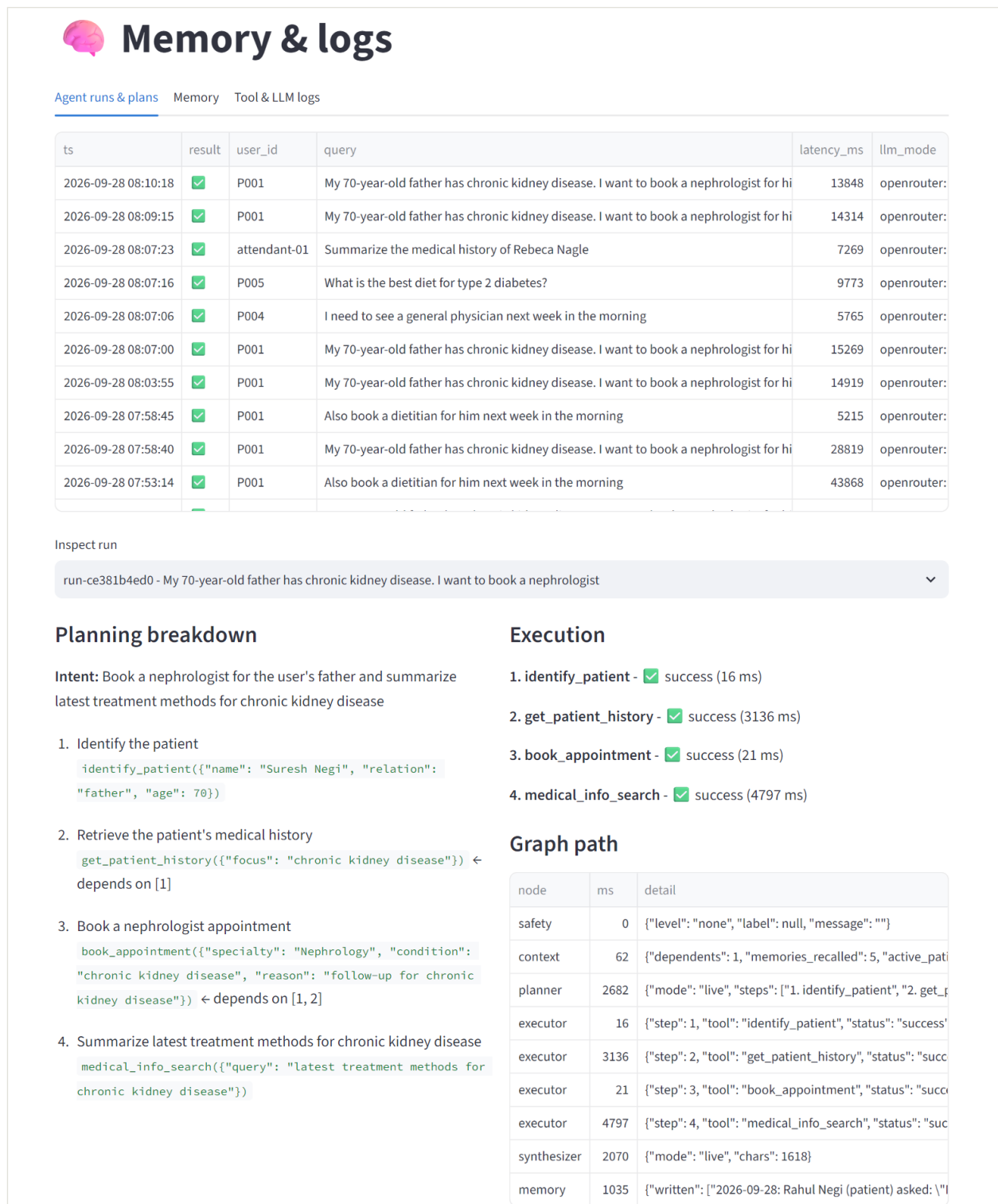


Figure 4. Memory & logs: planning breakdown, execution status, graph path with per-node timings, and memory operations for one run.

8. Guard-rails

Scenario	Behaviour
"My father has chest pain and is sweating and short of breath"	Emergency notice first (call 112 / 911); the information search is deferred.
Patient asks for another patient's history	identify_patient refuses: not self or a linked dependent; dependent steps are skipped and explained.
Patient asks to update their own record	Plan repaired: staff-only tool removed, clarification returned (attendant or doctor required).
Same booking requested twice	Existing appointment returned; no double booking.
Two claims on one slot	Conditional UPDATE: exactly one wins.
No slot in the requested window	Booking fails gracefully with the nearest alternatives.
Provider stalls or errors	Hard deadline → logged fallback to the heuristic path; the user still gets an answer.



Scenario tester

Each scenario runs in a fresh conversation thread as the listed user, exercising planning, tools, memory and guard-rails.

Scenarios

Capstone reference scenario x
Booking with date/time preference x
Pure information request x
Attendant history summary x
⌵

Or add a custom query (runs as Rahul Negi)

▶ Run selected

Attendant history summary done

scenario	user	query	plan
Capstone reference scenario	Rahul Negi	My 70-year-old father has chronic kidney disease. I want to book a nephrologist for hi	ident
Booking with date/time preference	Anjali Mehra	I need to see a general physician next week in the morning	ident
Pure information request	David Thompson	What is the best diet for type 2 diabetes?	medi
Attendant history summary	Front-desk Attendant (Asha)	Summarize the medical history of Rebeca Nagle	ident

run_success is False when any step failed or was skipped - which is the expected behaviour for the guard-rail scenarios.

> Capstone reference scenario - 4/4 steps

> Booking with date/time preference - 3/3 steps

> Pure information request - 1/1 steps

Figure 5. Scenario tester: curated scenarios (including guard-rails) run end to end in fresh threads.

9. LLMOps: evaluation and monitoring

Method. Golden sets cover 15 planner queries across patient, caregiver and attendant roles; 5 reference summaries (3 taken verbatim from records.xlsx); 8 medical QA pairs with listed key facts; 12 retrieval queries; and 8 booking scenarios plus idempotency and race checks. LangChain's **QAEvalChain** grades summaries and answers when an LLM is configured. Reference-based metrics always run: embedding similarity, ROUGE-L, token F1, key-fact coverage, context recall (key facts present in the retrieved passages) and groundedness (share of answer sentences supported by a retrieved passage at cosine ≥ 0.6). Evaluation bookings are rolled back.

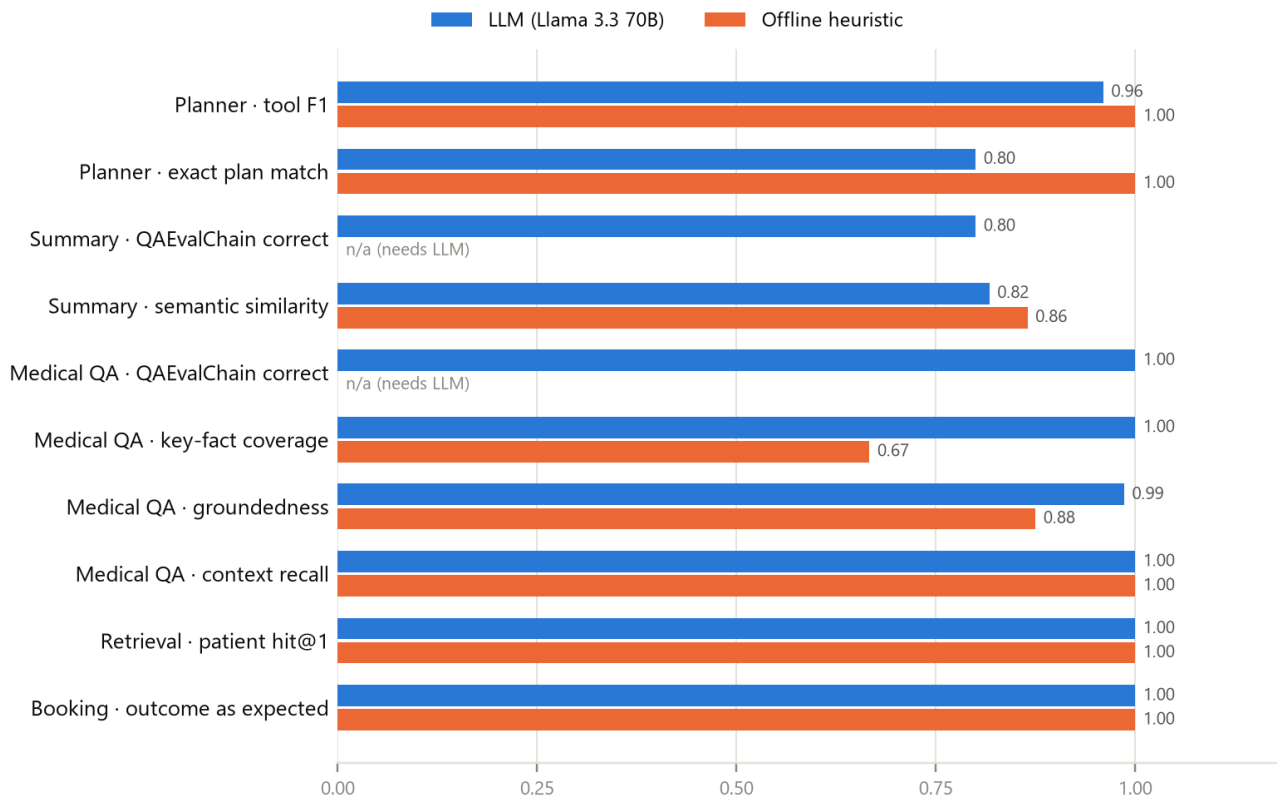


Figure 6. Evaluation results: live LLM vs offline heuristic mode (same golden sets).

Detailed results

Module	Metric	Offline	LLM
planner	Tool-selection F1	1.00	0.96
planner	Exact plan match	1.00	0.80
planner	Specialty accuracy	1.00	1.00
summary	QAEvalChain correct	–	0.80
summary	Semantic similarity	0.86	0.82
summary	ROUGE-L recall	0.75	0.53
medical qa	QAEvalChain correct	–	1.00
medical qa	Key-fact coverage	0.67	1.00
medical qa	Context recall	1.00	1.00
medical qa	Groundedness	0.88	0.99
retrieval	Patient memory MRR	1.00	1.00
retrieval	Knowledge hit@3	1.00	1.00
booking	Constraints satisfied	1.00	1.00
booking	Race safe	1.00	1.00

What the numbers say

- **Retrieval is not the bottleneck; generation is.** Context recall is 1.00 in both modes: every key fact was in the retrieved passages. The offline extractive summariser surfaced 67% of them; the LLM surfaced all of them, with groundedness rising from 0.88 to 0.99.
- **Offline summary scores are inflated.** The offline template echoes the recorded Summary field, which is also the reference, so its ROUGE-L is high by construction. The LLM paraphrases into structured sections, which lexical metrics penalise; QAEvalChain (0.80 correct) is the meaningful measure. The one miss was Rebeca Nagle, the most complex record.
- **Planner disagreements are judgment calls.** The heuristic planner's 1.00 is optimistic because it was written alongside the golden set. The LLM's three mismatches: it skipped the history lookup for two bookings where only a specialist (no condition) was named, and added a harmless list step before a reschedule.
- **MMR helped the offline path.** Replacing top-similarity sentence selection with Maximal Marginal Relevance ($\lambda = 0.35$) raised offline key-fact coverage from 0.56 to 0.69 at unchanged groundedness.

Monitoring

Every tool call, LLM call (live / fallback / offline), memory operation and agent run is logged to SQLite. The dashboard aggregates tool success rate and p50/p95 latency, per-module success, booking success and the LLM live/fallback mix, and can run the evaluation suite on demand.

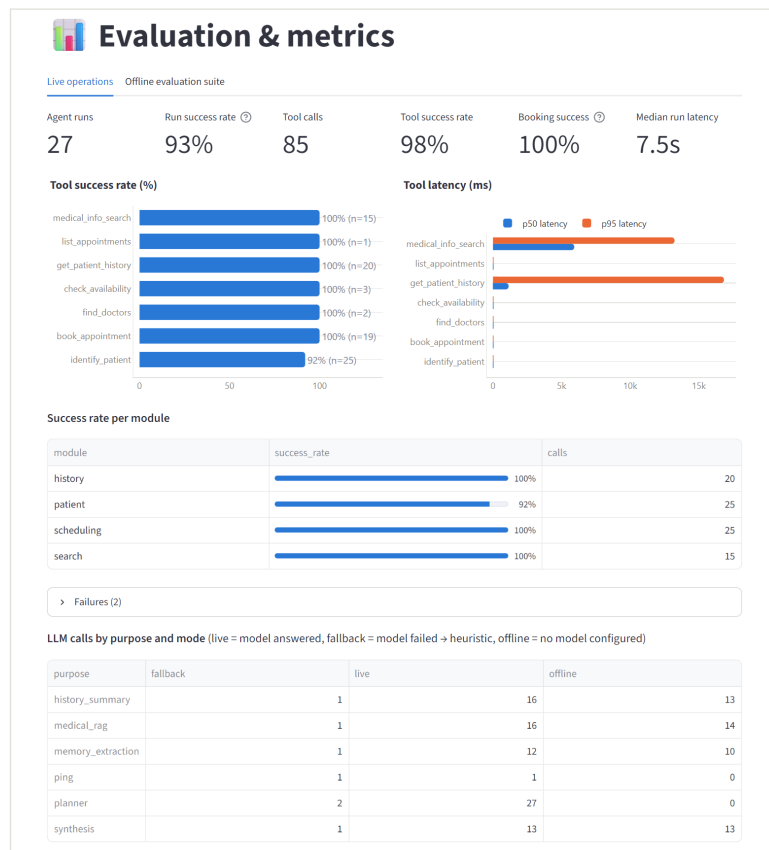


Figure 7. Live operations dashboard: run and tool success, latency, per-module success and LLM call modes.

10. Streamlit dashboard

Page	Purpose
Chat with the assistant	Live plan and execution trace per turn; example prompts per role
Scenario tester	Run curated or custom scenarios end to end (incl. guard-rails)
Patient view	Record, alerts, lab trends, appointments, memory; staff can update fields, add notes, upload PDFs and register patients
Doctor view	Calendar utilisation and upcoming patients with clinical briefing
Appointment tracker	Real-time (auto-refreshing) tracking, filters, cancellation
Medical information	Trusted-source search, latest retrieved answers, knowledge base
Evaluation & metrics	Tool/booking success, latency, LLM call modes, evaluation runs
Memory & logs	Memory traces, planning breakdowns, tool and LLM logs

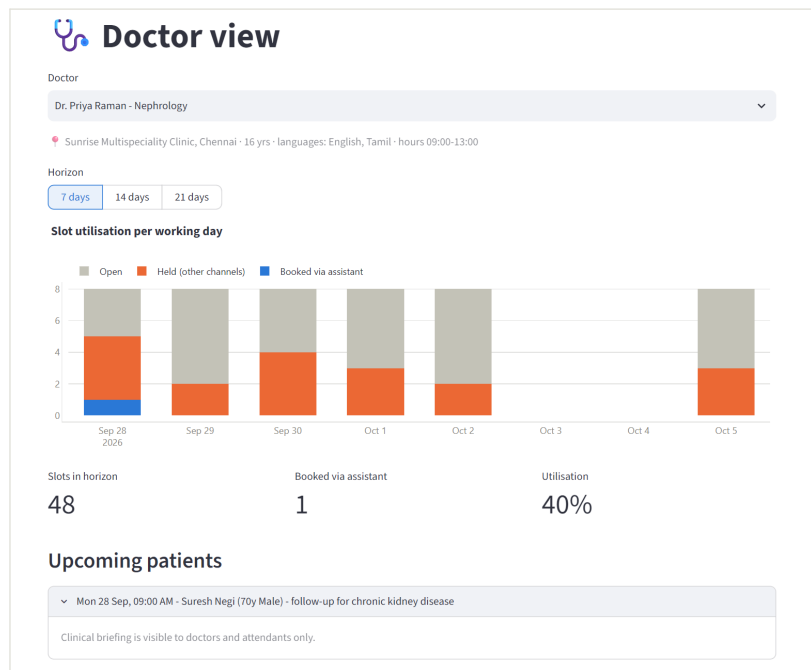


Figure 8. Doctor view: slot utilisation and the booked CKD patient.

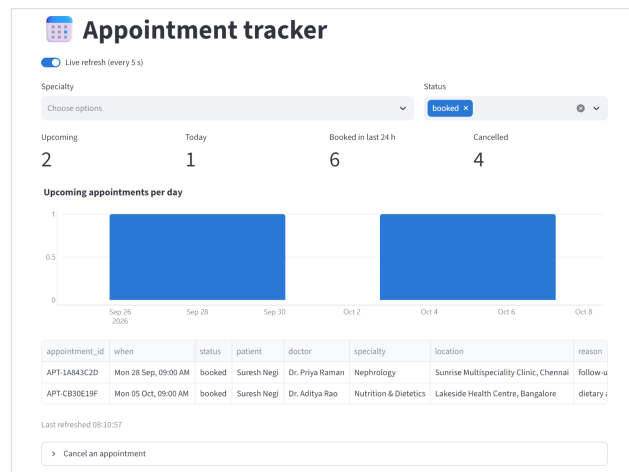


Figure 9. Appointment tracker with live refresh.

11. Engineering issues found and fixed

Testing against a live model surfaced problems that offline tests could not. Each was diagnosed from the logs and fixed in code:

Issue	Diagnosis	Fix
A follow-up turn hung for 14 minutes	The gateway kept a slow connection alive, so the client's 60 s read timeout never fired	Hard wall-clock deadline on every LLM call; timeouts fall back and are logged
Slow, variable LLM latency	Default provider routing: 1–5 s per call	Throughput-sorted provider routing: ~0.5 s per call
Redundant plan steps	Planner added find_doctors and check_availability before booking	Planner rule: booking already picks the earliest slot
Reply omitted patient context	Synthesis prompt did not ask for it	Patient-context section when history was retrieved
Doctor view showed 100% booked	Empty IDs load as NaN, which is truthy in Python	Explicit not-null check
Tests overwrote evaluation reports	Report folder not isolated in tests	Configurable REPORTS_DIR, pointed at a temp dir in tests

12. Limitations and future work

- **Synthetic scheduling data.** A production deployment would put a real calendar/EHR behind the same tool interfaces (e.g. FHIR Appointment, Slot and Patient resources).
- **Simulated authentication.** The acting-user selector stands in for real identity, consent records and audit logging (HIPAA / DPDP).
- **Source coverage.** WHO content arrives via web search; a curated guideline corpus (KDIGO, ADA, NICE) with freshness metadata would strengthen “latest treatment” answers.
- **Evaluation scale.** Golden sets are small; next steps are paraphrase expansion, an LLM-judge faithfulness score and tracking metrics across model versions in CI.
- **Human in the loop.** Caregiver-initiated bookings should require confirmation, and clinician sign-off should precede committing extracted notes.
- **Stale memories.** Interaction memories can outlive the state they describe (e.g. a later cancelled appointment); memories should be reconciled against the EHR.

Appendix: running the project

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env # add GROQ_API_KEY / OPENAI_API_KEY / OPENROUTER_API_KEY
streamlit run app.py # builds the EHR and indexes on first launch
python -m healthcare_agent.evaluation.runner # full evaluation suite
python -m pytest -q # 27 tests, isolated and offline
```

Repository: healthcare_agent/ (agent, tools, memory, evaluation), app.py + app_pages/ (dashboard), notebooks/ (executed walkthrough), tests/, docs/PROJECT_REPORT.md.

Educational prototype on sample and synthetic data. It does not diagnose or provide medical advice.